CS7DS2 Assignment - Week 6

Name: Kehan Liu ID:25334065

Q1

A

Full-batch Gradient Descent (GD) on the linear regression , I first generated the data according to the requirements of the problem. The sample size was set to m = 1000, and the elements in the feature matrix X followed a standard normal distribution $X_{ij} \sim N(0,1)$. Simultaneously, we defined the optimal parameters that we hoped the model would eventually learn $\theta^* = [3,4]^T$. Noise was also introduced.

```
np.random.seed ( 42 )
m = 1000
X = np.random.randn ( m , 2 )
theta_star = np . array ([ 3.0 , 4.0 ]). reshape ( 2 , 1 )
epsilon = np.random.randn ( m , 1 )
y = X @ theta_star + epsilon
theta_0 = np . array ([ 1.0 , 1.0 ]). reshape ( 2 , 1 )
alpha_const = 0.5
```

The mean squared error loss function for linear regression is defined as:

$$L(\theta)=\frac{1}{2m}||X\theta-y||_2^2$$

To minimize this loss function, we need to calculate θthe gradient of the loss function with respect to the parameters:

$$\nabla L(\theta)=\frac{1}{m}X^T(X\theta-y)$$

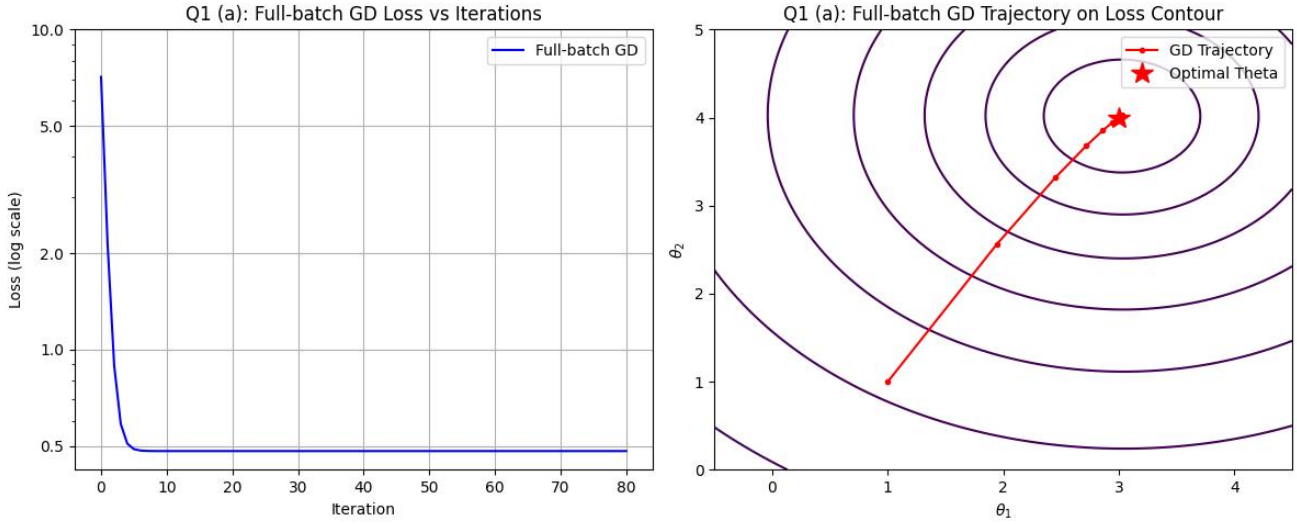
Substitute the X and Y values of all 1000 samples into the gradient formula above. α=0.5Update the parameters using a constant step size in the opposite direction of the gradient $\theta_{t+1}=\theta_t-\alpha\nabla L(\theta_t)$.

Repeat this process 80 times.

```
def full_batch_gd ( X , y , theta_init , alpha , iterations ):
    """Full Batch Gradient Descent"""
    theta = theta_init.copy ()
    history = [ theta.copy ()]
    losses = [ compute_loss ( theta , X , y )]
    for _ in range ( iterations ):
        grad = compute_gradient ( theta , X , y )
        theta = theta - alpha * grad
        history .append ( theta .copy ( ))
```

```
        losses . append ( compute_loss ( theta , X , y ))
    return np . array ( history ), np . array ( losses )
```

Finally, a graph $L(\theta_t)$showing the change with the number of iterations and the trajectory on the contour map were plotted $L(\theta_1,\theta_2)$, and the results are as follows.



As you can see from the loss curve on the left, in the initial 0 to 5 iterations, the loss value drops very rapidly, from the initial 8 to about 0.5, but in the subsequent 75 iterations, it hardly changes, showing a straight line that is very smooth and without any oscillations.

This $L(\theta_1,\theta_2)$is also verified by the trajectory $L(\theta_1,\theta_2)$on the contour map in the right figure. This trajectory is an almost straight line that points directly to the optimal solution. It can also be seen that the optimal solution was reached in the first 5 steps and then remained at the optimal solution.

I believe the main reason for this phenomenon is that Full-batch GD utilizes all samples to calculate the gradient in each update. Furthermore, the step size of α=0.5 is reasonable. Ultimately, the loss curve stabilizes at approximately 0.5, which is precisely the theoretical minimum error limit of the noise set during data $\frac{1}{2}\sigma^2$generation .

B

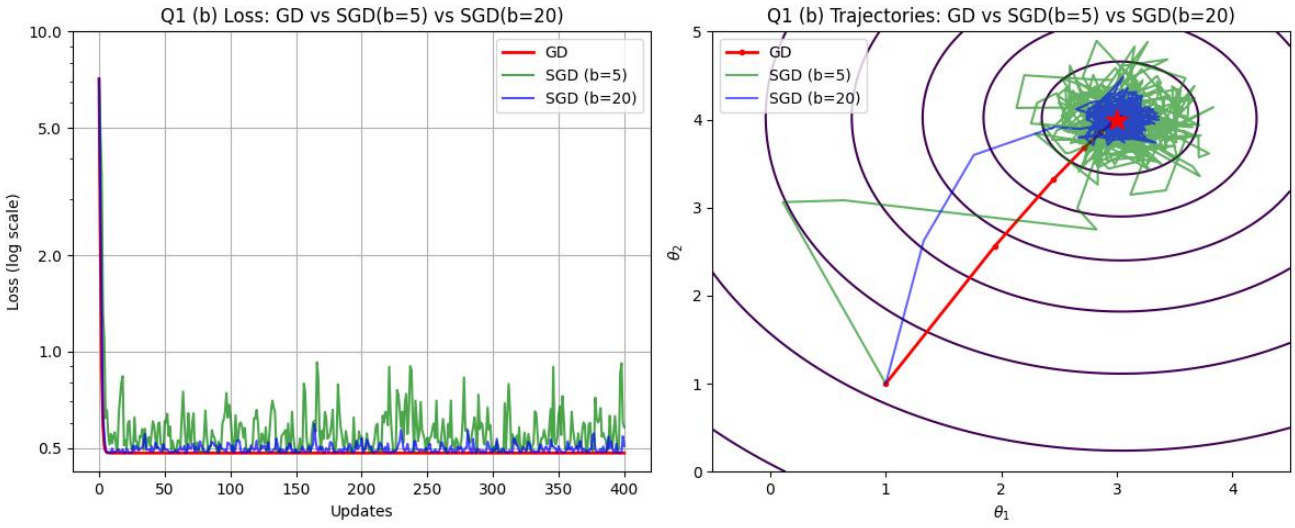
While Full-batch GD achieved excellent results in Q1.a, it was slow because it required calculating the entire sample each time. Mini-batch stochastic gradient descent effectively solves this problem by calculating the gradient using only a small subset of data to estimate the global gradient.

In implementation, before extracting samples at the start of each epoch, I use `np.random.permutation` to randomly shuffle the index of 1000 samples, and then slice the shuffled dataset sequentially according to the set batch size (5, 20). Local gradients are calculated, and parameters are updated $\theta_{t+1}=\theta_t-\alpha\nabla L_{batch}(\theta_t)$, for a total of 400 updates.

```
def mini_batch_sgd ( X , y , theta_init , alpha , batch_size , updates , method = 'constant' , beta = 0.9 , eps = 1e-8 ):
    """Mini-batch Stochastic Gradient Descent"""
    theta = theta_init.copy ()
```

```
history = [ theta.copy ()]
losses = [ compute_loss ( theta , X , y )] # Record full-batch loss

m = len ( y )

update_count = 0
while update_count < Updates :
    #Shuffle data before each epoch
    indices = np.random.permutation ( m )
    X_shuffled = X [ indices ]
    y_shuffled = y [ indices ]

    for i in range ( 0 , m , batch_size ):
        if update_count >= Updates :
            break
        X_batch = X_shuffled [ i : i + batch_size ]
        y_batch = y_shuffled [ i : i + batch_size ]

        if method == 'constant '
            grad = compute_gradient ( theta , X_batch , y_batch )
            theta = theta - alpha * grad
        history .append ( theta .copy ( ))
        losses . append ( compute_loss ( theta , X , y ))
        update_count += 1

return np . array ( history ), np . array ( losses )
```

Keeping the same constant learning rate α=0.5 as Q1.a, we plotted a comparison of the losses of GD, SGD (b=5) and SGD (b=20) and a contour plot of the trajectories of these three algorithms, as shown below.



The red line representing GD shows the same phenomenon observed in the previous problem: in the first few steps, the loss value of GD drops rapidly to 0.5 and reaches the optimal solution, and then remains almost unchanged in the subsequent 390+ updates.

Focusing on SGD(b=5) and SGD(b=20), in the loss curves of the left chart, both initially showed a trend almost identical to GD, rapidly decreasing to approximately 0.5. However, in subsequent updates, the loss curves of both exhibited significant oscillations. Specifically, the oscillations of SGD(b=5) were more pronounced than those of SGD(b=20), with the loss value of SGD(b=5) fluctuating between 0.5 and 1, while SGD(b=20) showed only minor oscillations.

In the right figure, the green trajectory of SGD (b=5) is zigzag and chaotic. In the first step, it jumps in the wrong direction, but in the first few steps, it still approaches the optimal solution in a zigzag manner. Near the optimal solution, it begins to jump back and forth repeatedly, forming a chaotic trajectory near the optimal point .

SGD (b=20) deviates from the red straight line in the early stages, its overall direction of travel is clearer than that of the green line . After reaching the vicinity of the optimal solution , although it also oscillates back and forth , its range of movement is smaller compared to the green line , and it stays close to the optimal solution.

I believe the reason for this phenomenon is that SGD (b=5) relies on only a very small number of samples for each update, resulting in large random noise in the gradient. However, when b=20, more samples participate in the update , the bias is smaller , and the gradient direction is more stable .

This demonstrates that in stochastic gradient descent, batch size can determine stability. While smaller batches result in faster single-step computation, the data is highly unstable; increasing the batch size effectively filters noise and achieves more accurate and stable convergence.

C

In Q1(b), we observed that mini-batch SGD causes trajectory oscillations due to differences in sample size. To suppress this noise and accelerate convergence, we introduce two advanced optimization strategies: the momentum-based NAG algorithm and the adaptive learning rate-based Adagrad algorithm.

The standard momentum method provides inertia by accumulating historical gradients, but it is prone to overshooting when approaching the optimum. NAG introduces a lookahead mechanism to address this.

In the code implementation, the current accumulated speed Vt is used to predict the possible next position:

$$\theta_{ahead}=\theta_t-\beta v_t$$

Pre-calculate the gradient , and then calculate the mini-batch gradient at the predicted location.θ_{ahead}

$$g_{ahead}=\nabla L_{batch}(\theta_{ahead})$$

By combining historical velocity and predicted gradient, the actual update step size is calculated, and the parameters are updated accordingly:

$$v_{t+1}=\beta v_t+\alpha g_{ahead}$$

$$\theta_{t+1}=\theta_t-v_{t+1}$$

This means that if an overshoot occurs next , NAG can know in advance and slow down in time.

```
        elif method == 'NAG' :
            theta_ahead = theta - beta * v
            grad_ahead = compute_gradient ( theta_ahead , X_batch , y_batch )
            v = beta * v + alpha * grad_ahead
            theta = theta - v
```

In contrast, Adagrad addresses the oscillation problem through an adaptive learning rate. Adagrad records the sum of squared gradients for each parameter throughout its history and adjusts the learning rate accordingly .

In its implementation, Adagrad introduces a state variable with the same dimension as the parameters G_t, accumulating the square of the current gradient in each iteration:

$$G_t=G_{t-1}+g_t\odot g_t$$

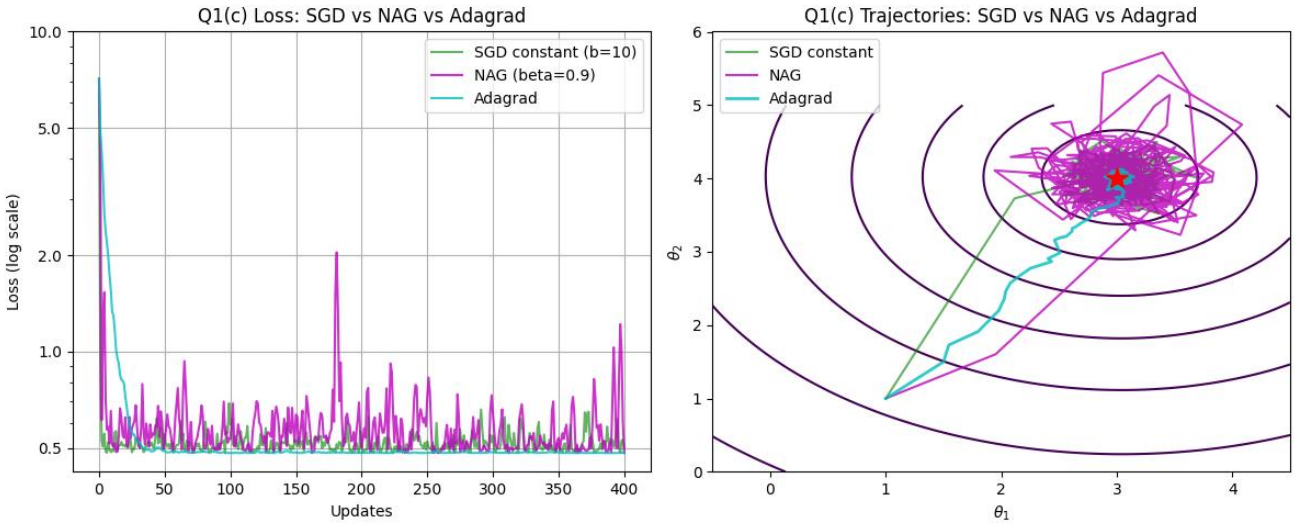
And the initial learning rate is scaled using cumulative variables:

$$\theta_{t+1}=\theta_t-\frac{\alpha}{\sqrt{G_t+\epsilon}}\odot g_t$$

This mechanism allows Adagrad to maintain a large step size in the early stages of optimization, while adjusting to a smaller step size as it approaches the optimal solution .

```
        elif method == 'Adagrad' :
            grad = compute_gradient ( theta , X_batch , y_batch )
            G += grad ** 2
            theta = theta - ( alpha) / np.sqrt ( G        + eps )) * grad
```

Maintaining the same constant learning rate $\alpha=0.5$ as Q1.a, plot the loss comparison graphs of SGD (b=10) , NAG (βb=0.9) , and Adagrad , as well as the contour plots of the trajectories of these three algorithms, as shown below.



Adagrad (cyan line) performs best , outperforming the other two algorithms in both the loss graph and trajectory graph . Firstly, in the loss graph, after reaching the theoretical limit of approximately 0.5 in the early stages, it remains relatively stable with very small fluctuations in subsequent updates. While its trajectory initially approaches the optimal solution with some twists and turns, the oscillations are minimal once it gets close, and the trajectory remains very close to the optimal solution.

SGD (green line) shows oscillations in the loss curve, but the oscillation amplitude is significantly smaller than that of NAG. Similarly, in the trajectory graph, SGD approaches the optimal solution relatively quickly and smoothly in the early stage, but oscillates near the optimal solution, but the oscillation amplitude is smaller than that of NAG.

NAG (purple line) performed the worst ; surprisingly, it exhibited extremely violent oscillations. In the trajectory graph, its trajectory formed a huge and chaotic cluster , showing severe oscillations . In the loss graph, its error frequently spiked suddenly to 1.0 or even higher , demonstrating extreme instability .

I believe the reason for this is that the value βis too αlarge. The algorithm retains 90% of the speed, which leads to it becoming increasingly faster . Moreover, for NAG, α0.5 is too large, resulting in the overshoot phenomenon .

This indicates that NAG must be paired with a smaller learning rate when introducing momentum , while adaptive algorithms like Adagrad exhibit strong robustness due to their mechanism of automatically adjusting the step size.

Q2

A

In this problem, the target model is [model name] $\hat{y}(u;x)=x_2 \tanh(x_1 u)$. First, let's analyze this model. The formula uses the classic activation function in neural networks. It tanhcontrols the linear input signal $x_1$within the interval [-1, 1].

It is precisely because of tanhthe introduction of this feature that the loss function J(x)is no longer the regular "bowl shape" of Q1, but has become a complex "canyon".

The data generation process is almost identical to that of Q1, so I won't go into too much detail here.

To achieve full-batch GD, we first derive the gradient of the loss function with respect to the sum $x_1$of parameters x_2:

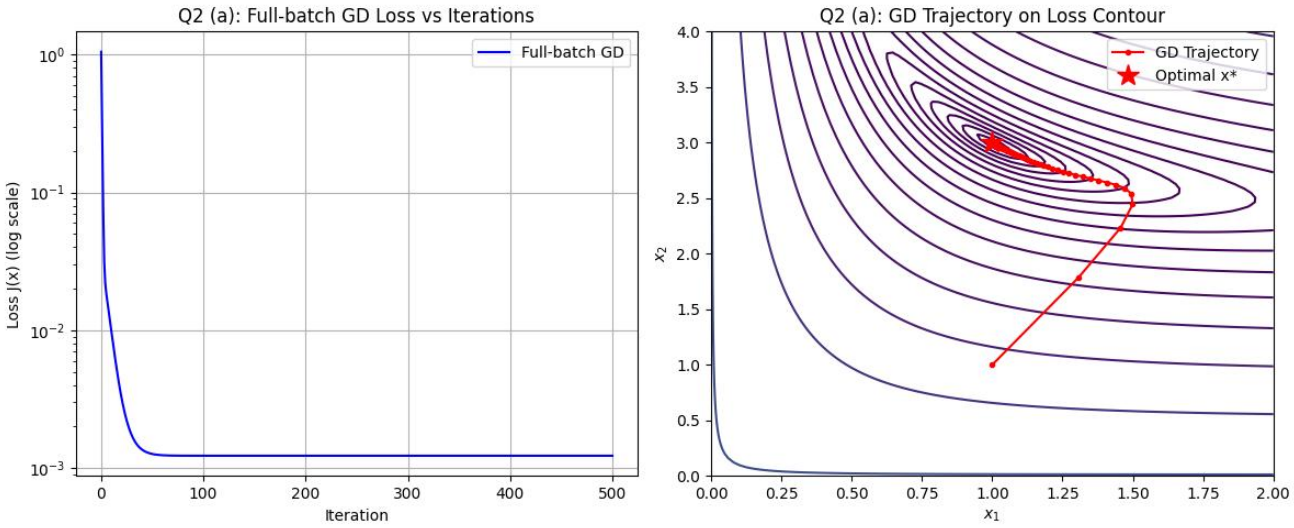
Find the partial derivative with respect to :x_2 $\frac{\partial J}{\partial x_2}=\frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)}-y^{(i)})\cdot \tanh(x_1u^{(i)})\frac{\partial J}{\partial x_2}$

Find the partial derivative with respect to :x_1 $\frac{\partial J}{\partial x_1}=\frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)}-y^{(i)})\cdot x_2\cdot (1-\tanh^2(x_1u^{(i)}))\cdot u^{(i)}$

In the code implementation, the algorithm iterates through all 1000 samples in each iteration to calculate the precise gradient mentioned above. And based on $x_{t+1}=x_t-\alpha\cdot \nabla J(x_t)$the updated...

```
def full_batch_gd_q2 ( u_data , y_data , x_init , alpha , iterations ):
    x_val = x_init.copy ()
    history = [ x_val.copy ()]
    losses = [ compute_loss_q2 ( x_val , u_data , y_data )]

    for _ in range ( iterations ):
        grad = compute_gradient_q2 ( x_val , u_data , y_data )
        x_val = x_val - alpha * grad
        history .append ( x_val .copy ( ))
        losses . append ( compute_loss_q2 ( x_val , u_data , y_data ))

    return np . array ( history ), np . array ( losses )
```

Run 500 iterations with a constant step size of 0.75, and plot $J(x_t)$the loss map and $J(x_1,x_2)$contour trajectory map as shown below.



The loss curve on the left shows that the model behaves very smoothly during iterations. Unlike Q1 linear regression , which quickly reaches the optimal solution within 5 iterations , the loss curve of

Q2 drops rapidly in the initial stage (the first 10 iterations) , and then the rate of decline slows down between 10 and 50 iterations . The loss value eventually converges to approximately $10^{-3}$the order of magnitude of the optimal solution after about 50 iterations .

The contour map on the right shows that the model's terrain is a winding "canyon." x_2The slope in one direction is steeper than the other, which is gentler in comparison .x_1

Initially, the model moves rapidly to the upper right, entering the valley . It then turns at the valley floor and moves along the valley floor towards the optimal solution . Finally, it reaches the optimal solution.

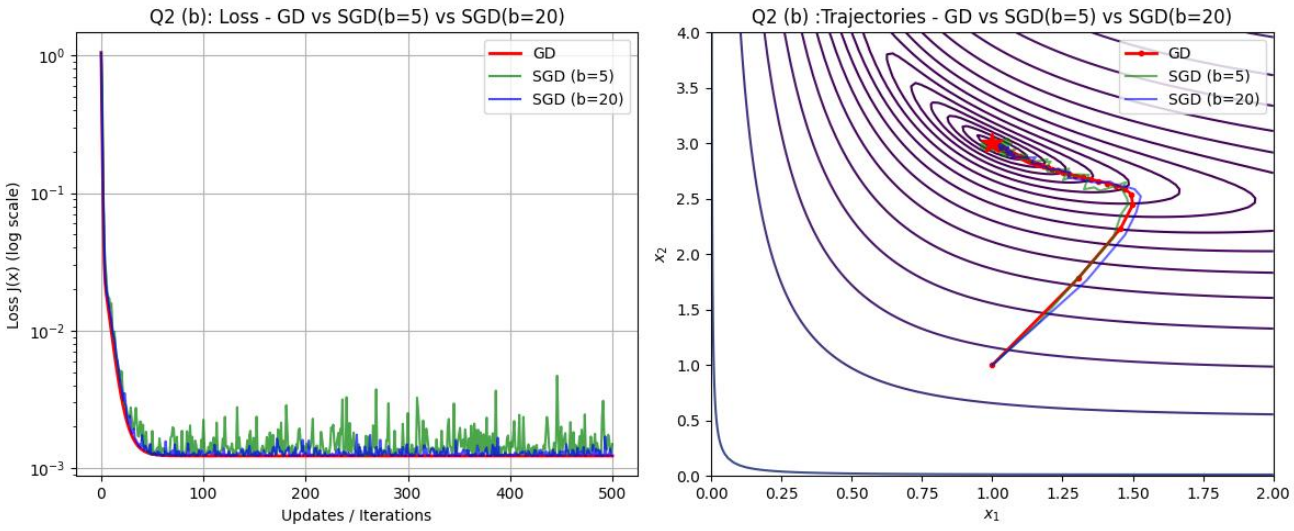
The results in Q2.a demonstrate that even with nonlinear models and complex terrain, full-batch gradient descent can still find the optimal solution by calculating all the data. Moreover, it is highly stable.

B

Similarly, to reduce computational costs, instead of iterating through all 1000 samples each time, we randomly select a small batch of samples (b=5 or b =20) to estimate the gradient.

mini-batch stochastic gradient descent is the same as in Q1.b, so it will not be elaborated on here.

Maintaining the same constant learning rate α=0.75 as Q2.a, we plotted a comparison of the losses of GD, SGD (b=5), and SGD (b=20) and a contour plot of the trajectories of these three algorithms, as shown below.



In the loss graph on the left, we can see that after the small-batch SGD converged quickly to the bottom in the early stage , it exhibited high-frequency oscillations of varying degrees . The oscillation amplitude of SGD (b=5) was significantly larger than that of SGD (b=20), which was very unstable. In contrast, SGD (b=20), due to the increased sample size, greatly reduced the noise impact caused by the small sample size, and its oscillation amplitude was significantly lower, making its performance close to that of the best-performing GD.

Contour trajectory diagram (right)

The red line (GD) represents the optimal path and can be used as a standard for other algorithms.

The green line (b=5) almost overlaps with the red line in the stage before entering the valley. Both enter the valley quickly in the early stage. However, the green trajectory at the bottom of the valley shows a zigzag pattern, jumping back and forth between the two valley walls. Moreover, after approaching the optimal solution, it jumps back and forth near the optimal solution, showing a chaotic trajectory cloud.

the blue line (b=20) is almost the same as that of the green line, but to a different degree. The blue line is more stable, and although there are slight fluctuations, it closely follows GD's red baseline .

I believe this phenomenon is due to two main reasons. First, the sample size at b=5 is extremely small, resulting in random noise in the calculated local gradient. Second, the terrain of this model is more complex. In this canyon terrain, even a slight deviation in the gradient direction, coupled with an excessively large learning rate (0.75) , can easily cause the gradient to collide with the canyon wall. Subsequently , the direction of the next gradient will forcefully push it towards the other side wall, thus forming the zigzag trajectory shown in the figure .

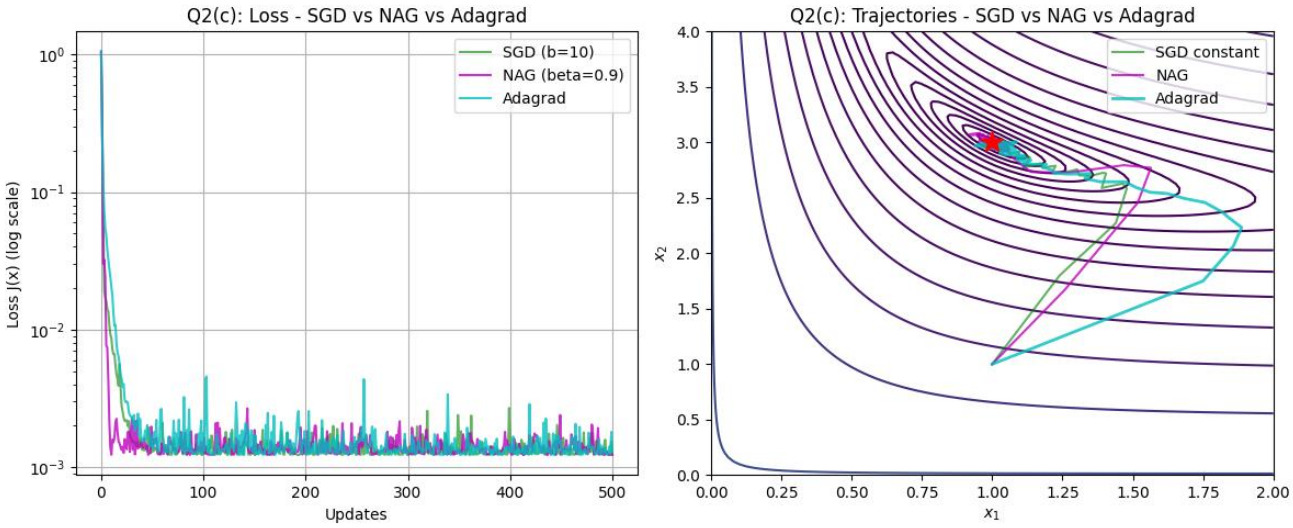
This illustrates that the more complex the terrain, the higher the accuracy of gradient estimation required by the algorithm. Using too small a batch size in SGD can cause the algorithm to bounce around on the canyon walls due to sampling noise, while appropriately increasing the batch size can effectively filter out noise and ensure stable convergence.

C

Add NAG and Adagrad for comparison

From a code implementation perspective, the logic of these two optimization algorithms is completely consistent with Q1(c) , so the implementation process will not be elaborated on here. The only difference is that the objective function is more complex.

Keeping the same learning rate $\alpha=0.75$ as Q2.a, we plotted the loss comparison graphs and contour plots of the trajectories of SGD (b=10) , NAG ($\beta=0.9$) , and Adagrad , as well as the loss comparison graphs of these three algorithms, as shown below.



The loss descent curves in the left figure show that all three algorithms exhibited an extremely rapid downward trend in the initial 0-30 updates, quickly approaching $10^{-3}$the magnitude of the loss. It's worth noting that NAG showed the fastest decline; in the first few iterations, NAG exhibited a vertical decline, significantly faster than the other two algorithms. However, after this point, all three algorithms displayed high-frequency oscillations, with almost indistinguishable amplitudes.

The right image shows contour trajectory maps; the three algorithms produced different trajectories.

SGD (green): It exhibits the most stable performance. After a slight initial dip, it quickly reaches the bottom, then oscillates along the bottom and gradually approaches the optimal solution.

NAG (purple): A slight offset and bounce occurred. Initially, it rushed further to the upper right than SGD, hitting the steeper contour edge on the right side of the valley, before turning and entering the valley floor. Near the optimum, the purple line exhibited the same zig-zag phenomenon. I believe this is due to NAG's momentum inertia. In the winding valley, inertia makes the algorithm turn more slowly. Therefore, it rushed further than SGD, but near the optimum, momentum again caused it to bounce back and forth at the valley floor.

Adagrad (cyan): It took a huge detour in the early stages. It completely deviated from the normal path into the valley and moved to the right. It didn't start to enter the valley normally until it was about X1=1.9 on the right side, where it also experienced oscillations.

I believe this is because, near the initial point [1, 1], due to tanhthe characteristics of the algorithm, the slope in the x_2 direction is steep, while the slope in the x_1 direction is relatively gentle. Therefore, in the initial iterations, a large sum of squared gradients accumulates in the x_2 direction, causing Adagrad to reduce the learning rate for x_2 , while the learning rate for x_1 remains large. Consequently, the algorithm moves rapidly horizontally, resulting in a rightward deviation. It only turns around after it has traveled a considerable distance, accumulated enough gradients in the horizontal direction, and the learning rates of both directions have rebalanced.

This suggests that for complex, nonlinear functions, mechanisms such as momentum and adaptive learning rates are likely to have a counterproductive effect.

Q3

A

In this problem, the objective function $f(x_1,x_2)=(1-x_1)^2+100(x_2-x_1^2)^2$is a classic function, and we have dealt with the same function in previous assignments.

As seen on the contour map, the function $x_2=x_1^2$forms a valley near this parabola.

The coefficient of the second term in the formula is as high as 100. This means that as long as the parameter deviates from the bottom parabola even slightly, the function value will rise rapidly.

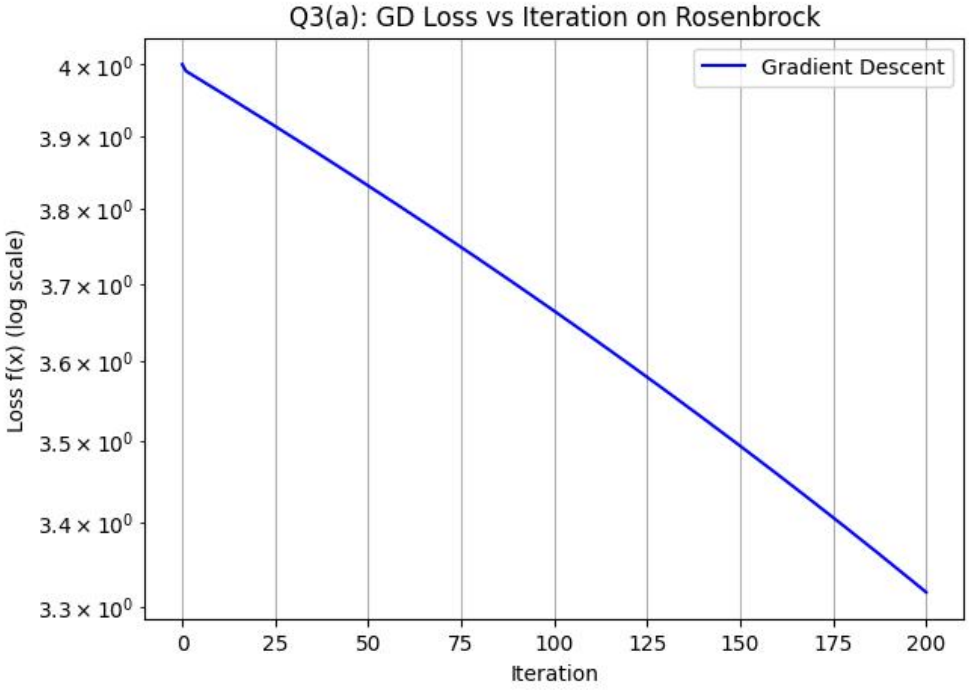
However, once the algorithm reaches the bottom of the valley, $(1-x_1)^2$the derivative of the first term becomes extremely weak. This means that the path along the valley to the optimal solution has an extremely gentle slope. x_1,x_2The gradients of the function differ greatly in direction.

First, I used the most basic gradient descent method. Unlike directly calculating the derivatives of Q1 and Q2, I used the finite difference method to approximate the gradient because the original function is too complex. By extracting two very close points, I used the formula $(f(x+h)-f(x-h))/(2h)$to approximate the gradient.

```
def finite_diff_grad ( f , x , h = 1e-5 ):
    """Compute gradient using central difference method"""
    grad = np.zeros_like ( x )
    for i in range ( len ( x )):
        x_plus = x.copy ()
        x_minus = x.copy ()
        x_plus [ i ] += h
        x_minus [ i ] -= h
        grad [ i ] = ( f ( x_plus ) - f ( x_minus )) / ( 2) * h )
    return grad
```

then updated in each iteration based on $x_{t+1}=x_t-\alpha\cdot\ \nabla f(x_t)$this. x_1,x_2The loss value is then calculated by substituting the first and second terms of the original function directly into the equation.

Run $f(x_t)$200 iterations with a step size of [value missing] $\alpha=10^{-3}$. The loss plot is shown below.



As can be seen, after 200 iterations, the loss value decreased from 4.0 to approximately 3.3. The blue line represents a downward trend, indicating that the decrease in loss value was very slow and limited . At the end of 200 steps, the algorithm still had not approached the global optimum (loss of 0).

I think this is mainly because the Rosenbrock function is very steep on both sides, so the learning rate is set very small $\alpha=10^{-3}$, resulting in slow movement.

This shows that GD using only the first derivative is inefficient, and it is difficult to move effectively under the constraint of small step sizes.

B

Similarly, the gradient is calculated using the finite difference method, reusing the code from Q3.b.

The core of Newton's method is using the Hessian matrix to calculate the curvature of the current terrain. When dealing with complex terrain like Rosenbrock, the diagonal elements of the Hessian matrix （$\frac{\partial^2 f}{\partial x_i^2}$） measure the steepness in a single dimension. On steep canyon walls, curvature is extremely high, while on flat valley floors, it is minimal. The off-diagonal elements of the Hessian matrix （$\frac{\partial^2 f}{\partial x_i \partial x_j}$） measure x_1和$x_2$the interaction between these elements.

In the Newton-Raphson update formula $x_{t+1}=x_t-\alpha H^{-1}g$, multiplying by is H^{-1}equivalent to performing an adaptive geometric distortion on the parameter space. It automatically reduces the step size in steep directions, increases the step size in flat directions, and forcibly twists the gradient direction that originally rushed towards the sidewalls to run along the valley floor.

Solving for the Hessian matrix still employs the finite difference approach. For the current parameter vector $x=[x_1,x_2]^T$, I set a very small value $h=10^{-5}$and define the unit vector of the coordinate axes. $e_1=[1,0]^Te_2=[0,1]^T$The Hessian matrix H is a 2x2 matrix, and the formula for calculating any element $H_{ij}$is as follows:

$$H_{ij}=\frac{\partial^2 f}{\partial x_i \partial x_j}\approx\frac{f(x+he_i+he_j)-f(x+he_i-he_j)-f(x-he_i+he_j)+f(x-he_i-he_j)}{4h^2}$$

```
def finite_diff_hess ( f , x , h = 1e-5 ):
    """Compute Hessian matrix using central difference method"""
    n = len ( x )
    hess = np.zeros ( ( n , n ) )
    for i in range ( n ):
        for j in range ( n ):
            if i == j :
                x_plus = x.copy ()
                x_minus = x.copy ()
                x_plus [ i ] += h
                x_minus [ i ] -= h
                hess [ i , i ] = ( f ( x_plus ) - 2 * f ( x ) + f ( x_minus )) / ( h ** 2 )
            else :
                x_pp = x.copy (); x_pp [ i ] += h ; x_pp [ j ] += h
                x_pm = x.copy (); x_pm [ i ] += h ; x_pm [ j ] -= h
                x_mp = x.copy (); x_mp [ i ] -= h ; x_mp [ j ] += h
                x_mm = x.copy (); x_mm [ i ] -= h ; x_mm [ j ] -= h
                hess [ i , j ] = ( f ( x_pp ) - f ( x_pm ) - f ( x_mp ) + f ( x_mm )) / ( 4) * h ** 2 )
    return hess
```

After obtaining the gradient vector g and the Hessian matrix h, the Newton direction can be obtained directly through matrix inversion p_t:

$$p_t=H_t^{-1}g_t$$

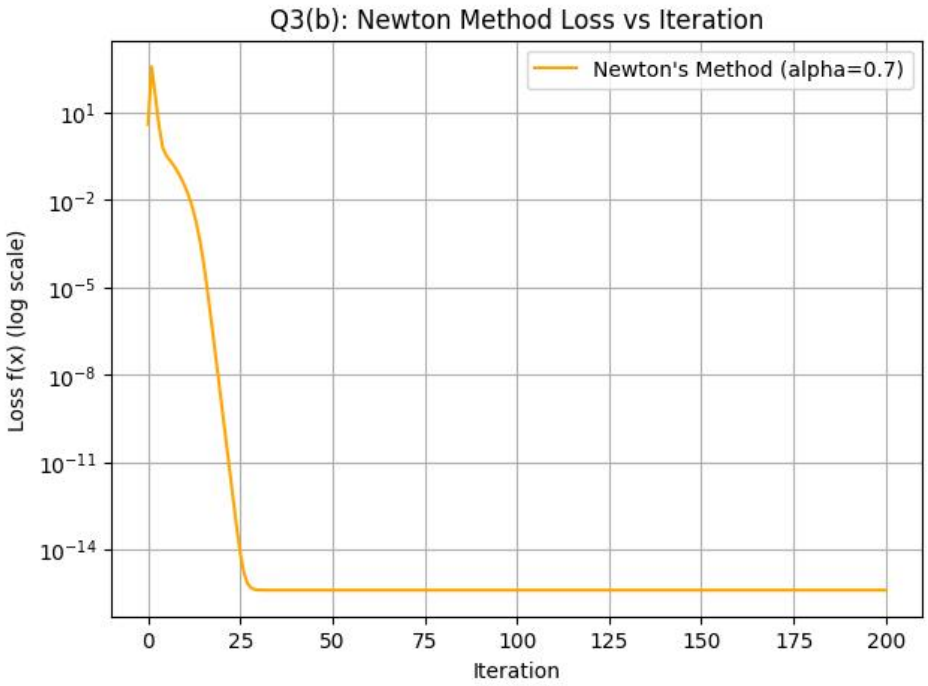
It uses the second derivative to scale and correct the direction of the gradient.

Then, in each iteration, $x_{t+1}=x_t-\alpha p_t$the position is updated based on this. Finally, $x_1,x_2$the value is directly substituted into the original function to calculate the loss.

```
def newton_rosenbrock ( x_init , alpha , iterations , lambda_reg = 1e-6 ):
    "Standard Newton"
    x = x_init.copy ()
    history , losses = [ x .copy()], [ rosenbrock ( x )]
    for _ in range ( iterations ):
        g = finite_diff_grad ( rosenbrock , x )
        H = finite_diff_hess ( rosenbrock , x )
        p = np . linalg . inv ( H ) . dot ( g )
        x = x - alpha * p
        history.append ( x.copy ( ) )
```

```
        losses . append ( rosenbrock ( x ))
    return np . array ( history ), np . array ( losses )
```

Run $f(x_t)$200 iterations with a constant scaling factor (α=0.7). The loss plot is shown below.



It can be observed that the orange curve, after an initial very short rise , exhibits a sharp downward trend . In just about 25 iterations, the loss value dropped from the initial value to approximately zero 10^{-15}, indicating that the algorithm found the optimal solution very quickly and without any oscillations, demonstrating remarkable stability.

This is because H^{-1}it automatically reduces the step size in steep directions and increases it in gentle directions, making the learning rate very stable even with a large learning rate . Furthermore, it forcibly aligns the update direction with the bottom of the banana valley parabola, directly pointing to the global optimum. This makes Newton's method not only stable but also fast.

However, it is worth noting that the loss value increased in the initial stage. I think this is because the local second-order Taylor expansion at the initial point (-1, 1) does not fully match the real global terrain. The standard Newton method moved an incorrect step size in the first step, causing this step to deviate from the bottom.

C

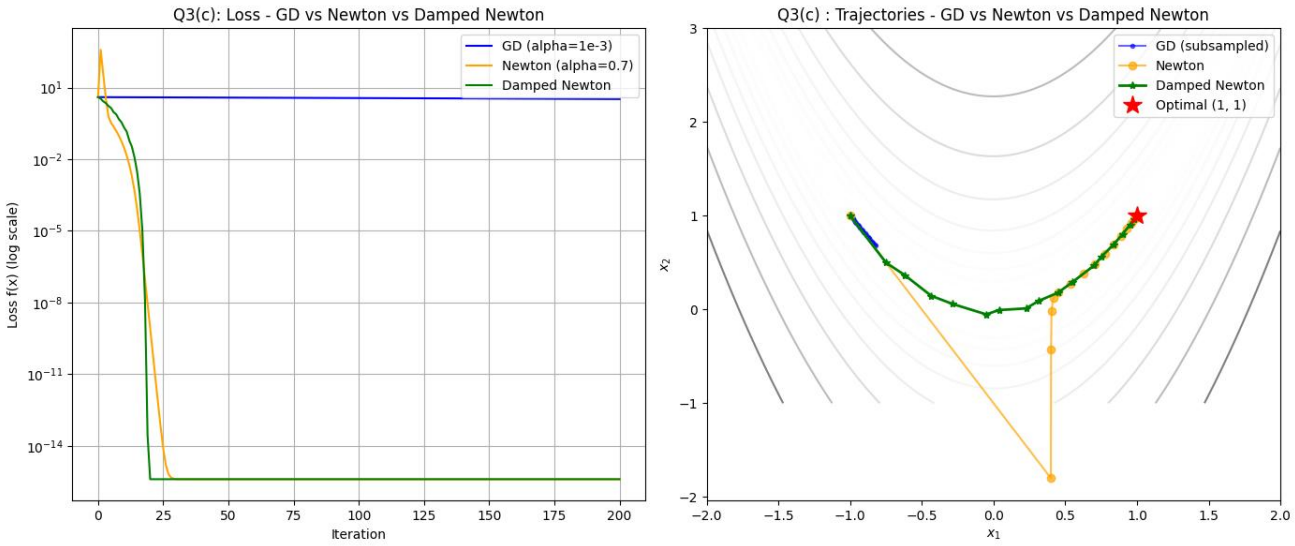
The standard Newton method performs well in Q3.b, but when the local second-order Taylor expansion does not conform to the actual terrain, the standard Newton method yields an overly aggressive or inappropriate update step, which causes the loss value to increase. The damped Newton method is a further optimization of the Newton method to address this problem.

In terms of implementation, the solution for the Hessian matrix and the gradient is consistent with the standard Newton method. The code from the previous question is reused here, and the solution for the Newton direction is also completely consistent with the previous question.

Unlike the standard Newton's method, the damped Newton's method does not directly accept an initial step size. Instead, it uses a loop to find the optimal step size, starting with the full step size by default α=1.0. If the step size is too large, causing the updated point to be larger than the current point, the algorithm multiplies the step size by a shrinkage factor rho（0.5）. When the backtracking search reaches its maximum number of attempts without success, the algorithm abandons the current second-order direction and performs a first-order gradient descent with a very small step size.

```
def damped_newton_rosenbrock ( x_init , alpha_0 = 1.0 , rho = 0.5 , max_k = 10 , iterations = 200 , lambda_reg = 1e-6 ):
    "Damped Newton"
    x = x_init.copy ()
    history , losses = [ x .copy()], [ rosenbrock ( x )]

    for _ in range ( iterations ):
        f_t = rosenbrock ( x )
        g_t = finite_diff_grad ( rosenbrock , x )
        H_t = finite_diff_hess ( rosenbrock , x )
        # Solve for Newton direction p_t
        p_t = np . linalg . inv ( H_t ). dot ( g_t )
        alpha = alpha_0
        step_accepted = False
        # Backtracking line search
        for k in range ( max_k ):
            x_new = x - alpha * p_t
            if rosenbrock ( x_new ) < f_t :
                step_accepted = True
                break
            else :
                alpha *= rho
        # Alternative fallback solution
        if not step_accepted :
            x_new = x - 1e-4 * g_t
        x = x_new
        history.append ( x.copy ( ) )
        losses . append ( rosenbrock ( x ))

    return np . array ( history ), np . array ( losses )
```

After 200 iterations, a comparison chart of the logarithmic loss of GD, Newton's method, and damped Newton's method, as well as contour plots of the trajectories of GD (subsampling display), Newton's method, and damped Newton's method are generated, as shown below.



Comparison of loss reduction curves (left figure)

GD (blue line): Due to the extremely small learning rate, the blue line is almost a horizontal line, showing almost no decrease in the 200 steps compared to the other two algorithms .

Standard Newton's method (orange line): In the first few iterations, the loss value increases slightly. The second-order Taylor approximation of the standard Newton's method is local, which proves that at the beginning, due to the mismatch between the local curvature and the global terrain, the algorithm produces an incorrect update step size, causing the loss value to increase. However, in the following 25 steps, the loss value drops rapidly to approximately 10^{-15}.

Damped Newton's method (green line): Performed best. The green line did not show the initial rise but instead dropped rapidly, smoothly reaching the optimal solution in about 20 steps.

Comparison of contour lines along the trajectory (right image)

Blue line (GD): Corresponding to the right figure, the blue line moves very slowly in 200 iterations, and is still a long way from the bottom.

Orange line (standard Newton's method): Because the local second-order Taylor expansion at the initial point (-1, 1) does not perfectly match the actual global terrain, the standard Newton's method first moves a huge step directly to the lower right, far deviating from the valley floor. It then re-enters the valley floor along a vertical direction. Finally, it moves along the valley floor to the lowest point.

Green line (damped Newton method): The green line smoothly and quickly finds the global lowest point along the optimal path to the bottom of the valley , without any oscillations, and is very stable.

This is thanks to the "Backtracking Line Search" introduced in the algorithm. When the algorithm attempts to use an incorrect update step size to approximate the standard second-order position at the initial position, the line search mechanism rejects it and shortens the step size,ensuring that the algorithm takes a valid step that strictly decreases the objective function.

The damped Newton method utilizes the step size check mechanism, completely avoiding the blindness problem of the standard Newton method, and is a highly efficient and stable optimization scheme.